BANGLADESH GOVT IT EXAM PREPARATION

Time Complexity & Space Complexity

Beginner → Advanced · বাংলায় ব্যাখ্যা, English definitions, C/C++/Java examples, step-by-step visualizations, 46 explained MCQs এবং Full Practice Test।

8 Chapters | 46 MCQs | Sorting Animation | 100% Offline

সূচিপত্র / Table of Contents

1. ১. ভিত্তি: Algorithm, Analysis ও Input Size
2. ২. Asymptotic Notation: Big O, Ω, Θ, o, ω
3. ৩. Common Time Complexities ও Growth Chart
4. ৪. Code Analysis: Statement, Loop, Recursion
5. ৫. Space Complexity: Input, Auxiliary, Stack, Trade-off
6. ৬. Important Algorithms: Search, Sort, Fibonacci, Array & String
7. ৭. Bangladesh Government IT Exam & Interview Preparation
8. ৮. Final Revision Sheet — One-Page Exam Checklist

Appendices: 46 MCQs এবং Answer Key with explanations.

> PDF একটি Static Reading & Printing Version। সব Interactive Features Offline HTML-এ পাওয়া যাবে।

১. ভিত্তি: Algorithm, Analysis ও Input Size

> **Learning outcome:** Algorithm কী, কেন বিশ্লেষণ করি, n কী বোঝায়, এবং Time–Space Complexity-এর সঠিক পার্থক্য বুঝতে পারবেন।

১.১ Algorithm কী?

English exam definition: *An algorithm is a finite, well-defined sequence of unambiguous steps to solve a problem or compute a result.*

বাংলায়: একটি নির্দিষ্ট সমস্যা সমাধানের জন্য পরিষ্কার, সুনির্দিষ্ট এবং সসীম সংখ্যক ধাপের পদ্ধতিকে **Algorithm** বলে। উদাহরণ—ক্লাসের সর্বোচ্চ নম্বর বের করা: (১) প্রথম নম্বরকে সর্বোচ্চ ধরা, (২) বাকি নম্বরগুলোর সঙ্গে তুলনা করা, (৩) বেশি পেলে সর্বোচ্চ পরিবর্তন করা, (৪) শেষে ফল প্রকাশ করা।

একটি ভালো Algorithm-এর বৈশিষ্ট্য: Input (শূন্য বা একাধিক হতে পারে), Output (অন্তত একটি), Definiteness (প্রতিটি ধাপ স্পষ্ট), Finiteness (শেষ হবে), Effectiveness (ধাপ বাস্তবে সম্পাদনযোগ্য)। Correctness এবং Efficiency আলাদাভাবে বিচার করা হয়।

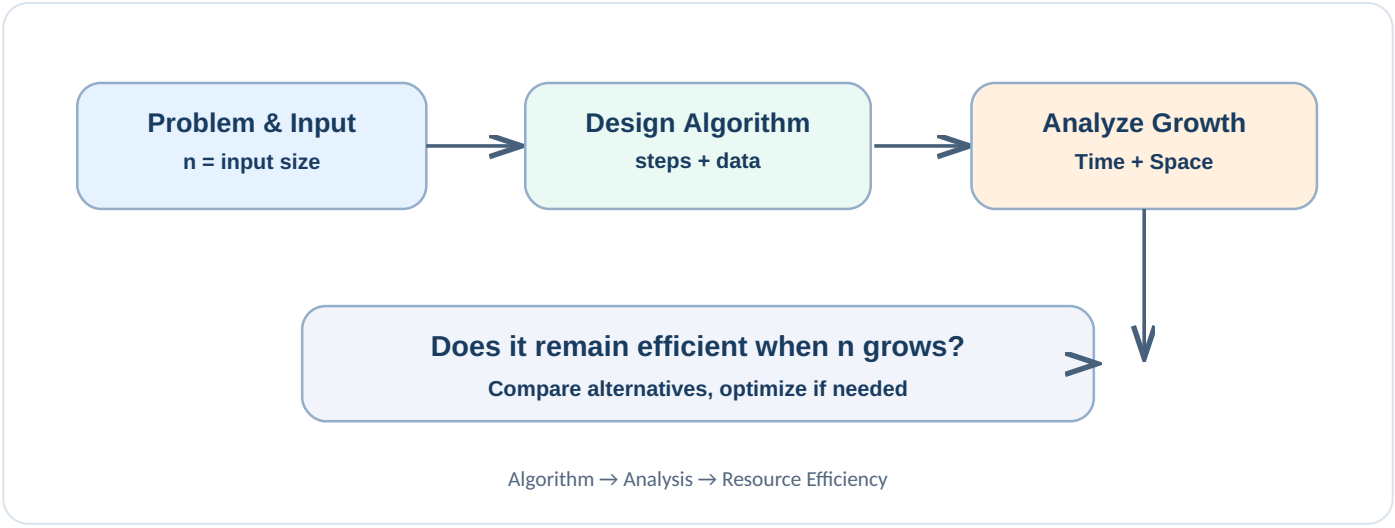
```
MAXIMUM(A, n)
    largest = A[0]
    for i = 1 to n-1
        if A[i] > largest
            largest = A[i]
    return largest
```

এখানে সর্বোচ্চ $n-1$টি comparison হয়; তাই Input বাড়লে কাজ লিনিয়ারভাবে বাড়ে।

১.২ Algorithm Analysis কী এবং কেন প্রয়োজন?

English exam definition: *Algorithm analysis studies the computational resources, especially time and memory, required by an algorithm as a function of input size.*

একই সমস্যার একাধিক সমাধান থাকতে পারে। কোনটি বড় Input-এর ক্ষেত্রে বেশি কার্যকর, তা বোঝার জন্য Time ও Memory-এর Growth পরীক্ষা করি। শুধু ঘড়িতে মেপে '২ সেকেন্ড' বলা যথেষ্ট নয়—কম্পিউটার, Compiler, Language, Network, Cache ইত্যাদি বদলালে সময় বদলায়। Complexity মূলত **Input Size পরিবর্তনের সঙ্গে Resource Requirement-এর বৃদ্ধির ধরণ** প্রকাশ করে।



১.৩ Input Size `n` কী?

- ৫টি সংখ্যার Array: `n = 5`; ১০ লাখের Array: `n = 1,000,000`।
- String Search-এ `n` String-এর দৈর্ঘ্য হতে পারে।
- Graph-এ প্রায়ই দুটি মাপ লাগে: `V` = Vertex ও `E` = Edge; তখন Complexity `O(V+E)` হতে পারে।
- Matrix-এ `r × c` Element; দুটি Dimension-ই আলাদাভাবে ধরতে হতে পারে।
- দুইটি ভিন্ন Array: দৈর্ঘ্য `n` ও `m` হলে Nested Loop-এর Complexity `O(nm)`, অকারণে `O(n²)` নয়।

১.৪ Time Complexity বনাম Space Complexity

দিক	Time Complexity	Space Complexity
প্রশ্ন	কতগুলো মূল Operation/Step?	কত Memory প্রয়োজন?
সাধারণ একক	Growth in steps	Growth in memory cells/bytes
উদাহরণ	Array scan → `O(n)` time	Input বাদে sum variable → `O(1)` auxiliary
গুরুত্বপূর্ণ সতর্কতা	Big O সরাসরি সেকেন্ড নয়	Auxiliary ও Total Space এক বিষয় নয়

লাইব্রেরি উদাহরণ (আগের আলোচনার ধারণা): সাজানো নয় এমন ১০০০টি বইয়ের মধ্যে একটি বই খুঁজতে শুরু থেকে একটি একটি করে দেখা (Linear Search) Worst Case-এ ১০০০টি পরীক্ষা করতে পারে, `O(n)`। সাজানো বইয়ের Catalog-এ Binary Search প্রত্যেক ধাপে সম্ভাব্য অংশ অর্ধেক করে, `O(log n)`। তবে Binary Search-এর জন্য সাজানো ও Random-Access উপযোগী তালিকা প্রয়োজন।

১.৫ Best, Average ও Worst Case

`A = [10, 20, 30, 40, 50]`-এ **Linear Search** দিয়ে `x` খুঁজলে:

Case	উদাহরণ	Comparisons	Tight bound
Best	`x = 10`	১	`O(1)`
Average*	লক্ষ্য সমান সম্ভাবনায় যেকোনো অবস্থানে	`(n+1)/2`	`O(n)`
Worst	`x = 50` অথবা অনুপস্থিত	সর্বোচ্চ `n`	`O(n)`

* **Average** নির্দিষ্ট Probability Distribution-এর ওপর নির্ভর করে; প্রতিটি Algorithm-এর জন্য 'মাঝের অবস্থান' ধরে নেওয়া যায় না।

বিশেষ পরীক্ষার ফাঁদ: `O()` নিজে Worst Case-এর সমার্থক নয়। এটি Mathematical Upper Bound; নির্দিষ্ট Best/Average/Worst-case Function-এর জন্যও Big O ব্যবহার করা যায়।

১.৬ একটি Code-এর Operation Count

```
int sum = 0;                // একবার initialization
for (int i = 0; i < n; i++) // loop n iterations
    sum += a[i];            // n বার addition/assignment
printf("%d", sum);          // একবার output
```

এখানে মোট কাজ `an+b` আকারের (a,b ধ্রুবক)। **Growth:** `O(n)` time। `sum` এবং `i` স্থির সংখ্যক Memory Cell ⇒ `O(1)` auxiliary space। Input Array `a` -সহ মোট Space ⇒ `O(n)` ।

২. Asymptotic Notation: Big O, Ω, Θ, o, ω

> **Learning outcome:** Bound-এর সঠিক অর্থ, Formal Definition, Constant ও Dominant Term বাদ দেওয়ার নিয়ম শিখবেন।

২.১ কেন Asymptotic Analysis?

ধরি দুটি Algorithm-এর Operation Count $T_1(n)=3n+5$, $T_2(n)=n^2+2$। ছোট n-এ ফল ভিন্ন হতে পারে, কিন্তু অনেক বড় n-এর জন্য Growth Trend-ই প্রধান। Asymptotic Analysis-এ স্থির Constant এবং Lower-order Terms Growth Classification পরিবর্তন করে না।

২.২ Big O — Upper Bound

English: $f(n) = O(g(n))$ *if there exist constants* $c > 0$ *and* n_0 *such that* $0 \le f(n) \le c\ g(n)$ *for every* $n \ge n_0$.

উদাহরণ: $f(n) = 3n + 5$। $n \ge 1$ হলে $3n+5 \le 8n$। কাজেই $3n+5 = O(n)$। **অর্থ:** পর্যাপ্ত বড় n-এ f(n) কোনো ধ্রুবক গুণিতক n-এর উপরে যায় না।

২.৩ Big Omega (Ω) — Lower Bound

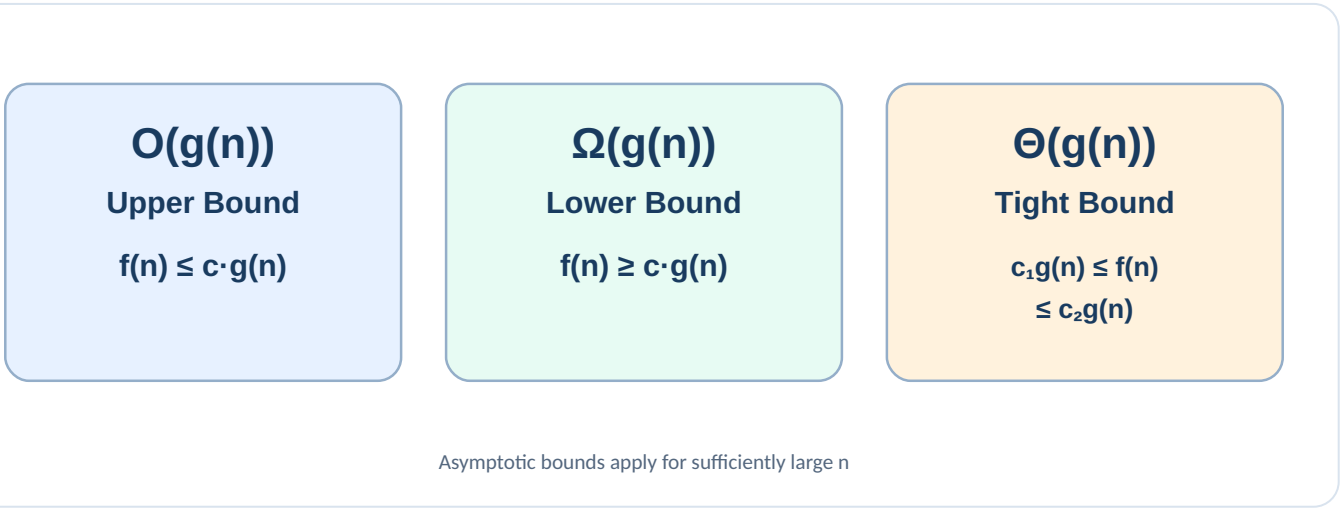
English: $f(n) = \Omega(g(n))$ *if for all sufficiently large n,* $f(n) \ge c\ g(n)$ *for some constant* $c > 0$.

$3n+5 \ge 3n$ ⇒ $3n+5 = \Omega(n)$। এর মানে **অন্তত সেই Growth Rate-এর সমান**।

২.৪ Big Theta (Θ) — Tight Bound

English: $f(n) = \Theta(g(n))$ *if* $f(n)$ *is both* $O(g(n))$ *and* $\Omega(g(n))$.

$3n+5 = \Theta(n)$; কারণ একই সঙ্গে Upper এবং Lower Bound n দিয়ে প্রতিষ্ঠিত। পরীক্ষায় 'Exact Asymptotic Order' জিজ্ঞেস করলে Θ উপযোগী। **সঠিকভাবে:** $n = O(n^2)$ সত্য, কিন্তু $n = \Theta(n^2)$ ভুল।



২.৫ Little o এবং Little omega (Advanced)

- `f(n) = o(g(n))` মানে f **strictly slower-growing**: `lim f(n)/g(n) = 0` (যখন সংশ্লিষ্ট Limit প্রযোজ্য)। যেমন `n = o(n²)`।
- `f(n) = ω(g(n))` মানে f **strictly faster-growing**: `lim f(n)/g(n) = ∞`। যেমন `n² = ω(n)`।
- `n` কিন্তু `o(n)` নয়; `n = Θ(n)`।

২.৬ Growth Simplification — পরীক্ষার Shortcut

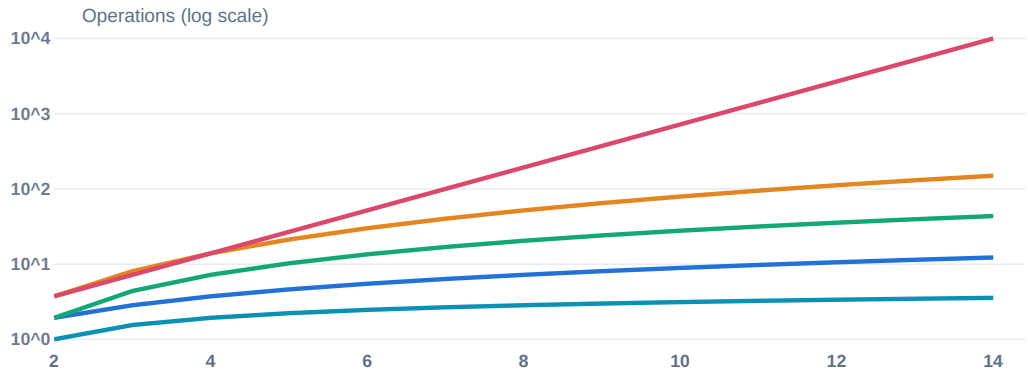
Operation Count	Tight asymptotic bound	কারণ
`100`	`Θ(1)`	Constant
`7n + 900`	`Θ(n)`	Constant, coefficient বাদ
`3n² + 8n + 1`	`Θ(n²)`	Highest-degree term
`n log₂n + 20n`	`Θ(n log n)`	Dominant term
`n³ + 2ⁿ`	`Θ(2ⁿ)`	Exponential eventually dominates
`log₂n` vs `log₁₀n`	উভয় `Θ(log n)`	Base পরিবর্তন Constant Factor

পরীক্ষায় Calculation: `T(n)=5n²+3n+7` ⇒ সবচেয়ে দ্রুত বাড়ে `n²` ⇒ **Θ(n²)**। কিন্তু সব ক্ষেত্রে সংখ্যাগত Runtime-ও বড় হবে, এমন দাবি করা যাবে না; Constant ও hardware গুরুত্বপূর্ণ।

<details><summary>Formal Bound প্রমাণ: `5n² + 3n + 7 = Θ(n²)`</summary> `n ≥ 1`-এ `5n² ≤ 5n²+3n+7 ≤ 15n²` (কারণ `3n ≤ 3n²`, `7 ≤ 7n²`)। অতএব `c₁=5`, `c₂=15`, `n₀=1` বাছলে দুটি Bound পূরণ হয়। </details>

৩. Common Time Complexities ও Growth Chart

> **Learning outcome:** সবচেয়ে সাধারণ আটটি Growth Class চিনবেন, তুলনা করবেন, Code-এর সঙ্গে মিলিয়ে দেখবেন।



Comparative growth (conceptual operation counts, logarithmic vertical scale)

৩.১ `O(1)` — Constant

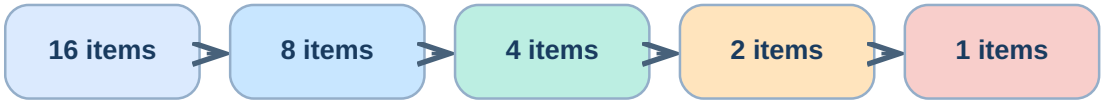
```
int x = a[2];          // valid array index access
int y = x + 5;
```

স্থির সংখ্যক Operation। Array-তে ৫টি বা ১০ লাখ Element থাকলেও নির্দিষ্ট Index Access-এর Asymptotic Cost `O(1)`।
সতর্কতা: Input-এর ওপর নির্ভর না করলেও বাস্তবে Operation Zero Time নয়।

৩.২ `O(log n)` — Logarithmic

```
while (n > 1) {
    n = n / 2;
}
```

Input প্রতি Iteration-এ অর্ধেক: `16 → 8 → 4 → 2 → 1` । ৪টি Halving; $\log_2 16 = 4$ । `1024 → 1` যেতে ১০টি Halving।
`O(log n)` সাধারণত Binary Search ও Balanced Tree Search-এ পাওয়া যায়।



16 → 8 → 4 → 2 → 1

4 halvings = $\log_2(16)$

Binary-search intuition: cut the remaining candidates by half

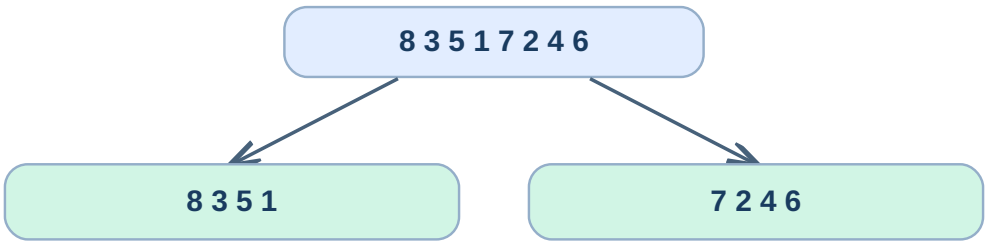
৩.৩ `O(n)` — Linear

```
for (int i = 0; i < n; i++) {
    printf("%d ", a[i]);
}
```

`n` Elements দেখলে `n` বার Visit করতে হয়। ৫ → ৫ iterations; ১০০ → ১০০ iterations। Loop variable fixed space ⇒ `O(1)` Auxiliary Space।

৩.৪ `O(n log n)` — Linearithmic

Merge Sort দুটি Half-এর Result Merge করে। Recursion depth প্রায় $\log_2 n$; প্রতিটি Level-এ মোট `n` Element Process ⇒ `O(n log n)`। Standard Merge Sort-এর Auxiliary Array `O(n)` Space নেয়।



$\log_2 n$ levels × n merge work per level

Time = Θ(n log n)

Merge Sort divides and then merges each level

৩.৫ `O(n²)` — Quadratic

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("*");
    }
}
```

প্রতি Outer Iteration-এ `n` টি Inner Iteration; মোট `n×n = n²` । `n=4 → 16`, `n=100 → 10,000` । কিন্তু **দুটি Nested Loop মানেই সর্বদা n^2 নয়**; Bounds পরীক্ষা করতে হবে।

৩.৬ `O(n³)` — Cubic

তিনটি স্বাধীন `n` -Iteration Loop হলে `n×n×n = n³` । উদাহরণ, Classical Triple-loop Matrix Multiplication (সমান আকারের n×n Matrix) `Θ(n³)` Arithmetic Operations।

৩.৭ `O(2ⁿ)` — Exponential

Naive Recursive Fibonacci-এর Recurrence `T(n)=T(n−1)+T(n−2)+Θ(1)` । এটি **$\Theta(\varphi^n)$** ($\varphi \approx 1.618$) Growth; `O(2ⁿ)` একটি বৈধ কিন্তু Loose Upper Bound। `2ⁿ` -এর নিখুঁত উদাহরণ হলো `n` টি Element-এর সব Subset Enumerate করা: **$\Theta(n \cdot 2^n)$** Output Cost হতে পারে যদি প্রতিটি Subset-এর Element-গুলোও ছাপানো হয়, এবং **$\Theta(2^n)$** Leaf/Subset Count।

৩.৮ `O(n!)` — Factorial

`n` টি ভিন্ন Item-এর সব Permutation-এর সংখ্যা `n! = n×(n−1)×…×1` । সব Permutation-এ nটি Item লেখা হলে Output Work `Θ(n·n!)` । Permutation Count `Θ(n!)` । `5! = 120`; `8! = 40320` । Exhaustive Permutation Search-এ এটি গুরুত্বপূর্ণ।

৩.৯ তুলনা: n বাড়লে কী ঘটে?

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
2	1	2	2	4	8	4	2
4	2	4	8	16	64	16	24
8	3	8	24	64	512	256	40,320
10	3.32	10	33.22	100	1,000	1,024	3,628,800
16	4	16	64	256	4,096	65,536	20,922,789,888,000

Typical growth ordering (large n, positive constants ignored): `1 < log n < n < n log n < n² < n³ < 2ⁿ < n!` । এই Ordering **Asymptotic**; ছোট n-এ মানের ক্রম ভিন্ন হতে পারে।

8. Code Analysis: Statement, Loop, Recursion

> **Learning outcome:** নতুন Code দেখে নিজের হাতে Count বের করবেন। প্রতিটি উদাহরণে Time ও Extra Space আলাদা ভাববেন।

৪.১ Simple Statements ও Conditionals

```
int a = 10;                  // 0(1)
int b = a * 4 + 2;           // 0(1)
if (x > 0)                  // constant-time comparison
    printf("positive");
else
    printf("non-positive");
```

স্থির সংখ্যক Primitive Operation ⇒ `0(1)` Time, `0(1)` Extra Space। `if` -এর Branch-এর Cost আলাদা হলে **Worst Case** সর্বাধিক Branch Cost; **Best Case** সবচেয়ে কম Branch Cost (condition evaluation-সহ)।

```
if (flag) {
    for (int i = 0; i < n; i++) work(); // 0(n)
} else {
    work();                             // 0(1)
}
```

Best `0(1)` ; Worst `0(n)` (ধরা হচ্ছে work() constant-time)। **Branch-এর Cost যোগ করবেন না**, একবারে একটি Branch চলে।

৪.২ Consecutive Loops: Add

```
for (int i = 0; i < n; ++i) std::cout << i;
for (int j = 0; j < n; ++j) std::cout << j;
```

`n + n = 2n` ⇒ `0(n)` Time। যদি প্রথম Loop n^2, দ্বিতীয় Loop n হয়, `n²+n` ⇒ `0(n²)` ।

৪.৩ Nested Loops: Multiply — তবে Bounds দেখুন

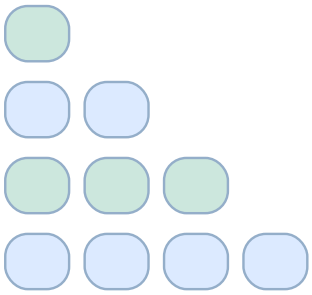
```
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++) {
        System.out.print("*");
    }
}
```

`n × n = n²` ⇒ `0(n²)` ; কিন্তু `j<n` -এর বদলে `j<m` হলে `0(nm)` ।

৪.৪ Triangular Loop

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        count++;
    }
}
```

Inner Iterations = `0+1+2+…+(n−1) = n(n−1)/2` ⇒ `0(n²)` । এটি Bubble/Selection Sort-এর Comparison Count বুঝতে সাহায্য করে।



Triangular loop: 0+1+…+(n−1) = n(n−1)/2

৪.৫ Independent Growth এবং Unusual Loops

```
for (int i=1; i<n; i*=2) work();   // 0(log n)
for (int i=n; i>0; i/=3) work();   // 0(log n)
for (int i=1; i*i<=n; i++) work(); // 0(√n) (safe arithmetic assumed)
```

```
for (int i=0; i<n; i++) {
    for (int j=1; j<n; j*=2) work();
}
```

Outer `0(n)`, Inner `0(log n)` ⇒ `0(n log n)` । আবার `for(i=1; i<n; i+=i)` অর্থাৎ `i` দ্বিগুণ ⇒ `0(log n)`; কিন্তু `i=0` থেকে `i+=i` শুরু করলে **Infinite Loop** হতে পারে।

৪.৬ Two-Pointer Loop: Nested হলেও O(n)

```
int j = 0;
for (int i = 0; i < n; ++i) {
    while (j < n && a[j] < a[i]) {
        ++j;
    }
}
```

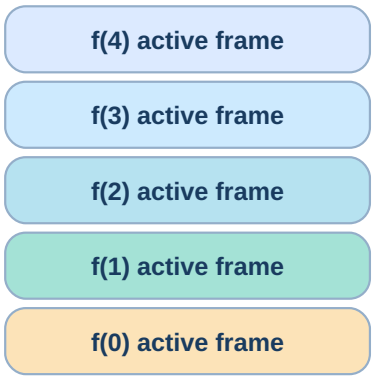
`j` কখনো Reset হয় না এবং সর্বোচ্চ `n` বার বাড়ে। Outer Loop n বার, Inner `while`-এর সব Iteration মিলিয়েও সর্বোচ্চ n ⇒ মোট `0(n)` (condition checks-সহ)। **Shortcut:** Loop-এর বডি কতবার **সর্বমোট** চলে, সেটি গণনা করুন।

৪.৭ Recursion ও Recurrence Relation

Recursion-এ Function নিজেকেই Call করে। **Base Case** না থাকলে থামবে না। Time Complexity বের করতে প্রতিটি Call কত কাজ করে এবং কয়টি Sub-call হয় তা দেখতে হয়। Space Complexity নির্ভর করে **Maximum Active Call-Stack Depth**-এর ওপর।

```
void countdown(int n) {
    if (n <= 0) return;
    printf("%d ", n);
    countdown(n-1);
}
```

`T(n)=T(n−1)+Θ(1)` ⇒ `Θ(n)` Time। একসঙ্গে n+1টি Call Frame ⇒ `Θ(n)` Stack Space।



Recursive call stack at the deepest point: f(4) → f(0)

৪.৮ Recurrence-এর আরও উদাহরণ

Recurrence	সমাধান	কোথায় দেখা যায়
`T(n)=T(n−1)+1`	`Θ(n)`	Countdown
`T(n)=T(n/2)+1`	`Θ(log n)`	Binary Search
`T(n)=2T(n/2)+n`	`Θ(n log n)`	Merge Sort
`T(n)=2T(n/2)+1`	`Θ(n)`	Full binary divide tree
`T(n)=T(n−1)+n`	`Θ(n²)`	Growing repeated work
`T(n)=T(n−1)+T(n−2)+1`	`Θ(φⁿ)`	Naive Fibonacci

Master Theorem (intro): `T(n)=aT(n/b)+f(n)`-এ `a≥1`, `b>1` এবং যথাযথ Regularity Conditions পূরণ হলে `n^(log_b a)`-এর সঙ্গে `f(n)` তুলনা করে Bound পাওয়া যায়। সব Recurrence-এ Master Theorem প্রযোজ্য নয়।

৪.৯ Common Mistakes / ভুলগুলো এড়িয়ে চলুন

1. `Big O = always worst case` — ভুল; Upper Bound ও Case Analysis আলাদা।
2. Consecutive Loop-এ Multiply করা — ভুল; Consecutive হলে **Add**, nested হলে সাধারণত **Multiply**, তবে shared pointers ও nonuniform bounds আলাদা।
3. Nested Loop দেখেই `O(n²)` বলা — ভুল; Inner Iterations যোগ করুন।

4. `O(n+m)`-কে `O(n)` ধরা — যদি m independent হয়, ভুল।
5. Log-এর Base নিয়ে অতিরিক্ত চিন্তা — Fixed-base Logs শুধু Constant Factor-এ আলাদা।
6. `T(n)=Θ(n²)` থেকে বাস্তব সময় 'নিশ্চয়ই n^2 সেকেন্ড' বলা — ভুল।
7. Recursive Algorithm-এ Call Stack বাদ দেওয়া — ভুল।
8. Binary Search Unsorted Data-তে চালানো — Standard Binary Search-এর Precondition লঙ্ঘন।
9. Mutable String Concatenation ও Immutable String Concatenation-কে একই Cost ধরা — ভুল; Representation গুরুত্বপূর্ণ।
10. Algorithm-এর Print/Output Work উপেক্ষা — Output Size নিজেই Lower Bound তৈরি করতে পারে।

<details><summary>পরীক্ষার ৫ ধাপের Decision Method</summary> (১) Input size ও একাধিক dimension চিহ্নিত করুন। (২) Basic Operations-এর সংখ্যা Count করুন। (৩) Conditional হলে Case পৃথক করুন। (৪) Loops/Recurrence সমাধান করুন। (৫) Leading Term রেখে Bound লিখুন এবং Memory আলাদাভাবে গণনা করুন। </details>

৫. Space Complexity: Input, Auxiliary, Stack, Trade-off

> **Learning outcome:** অতিরিক্ত Memory, Call Stack, In-place এবং Time–Space Trade-off সম্পর্কে সঠিক ধারণা অর্জন করবেন।

৫.১ Input Space বনাম Auxiliary Space

English exam definitions: *Input space is the storage occupied by the input. Auxiliary space is the additional working memory used by the algorithm, excluding its input.*

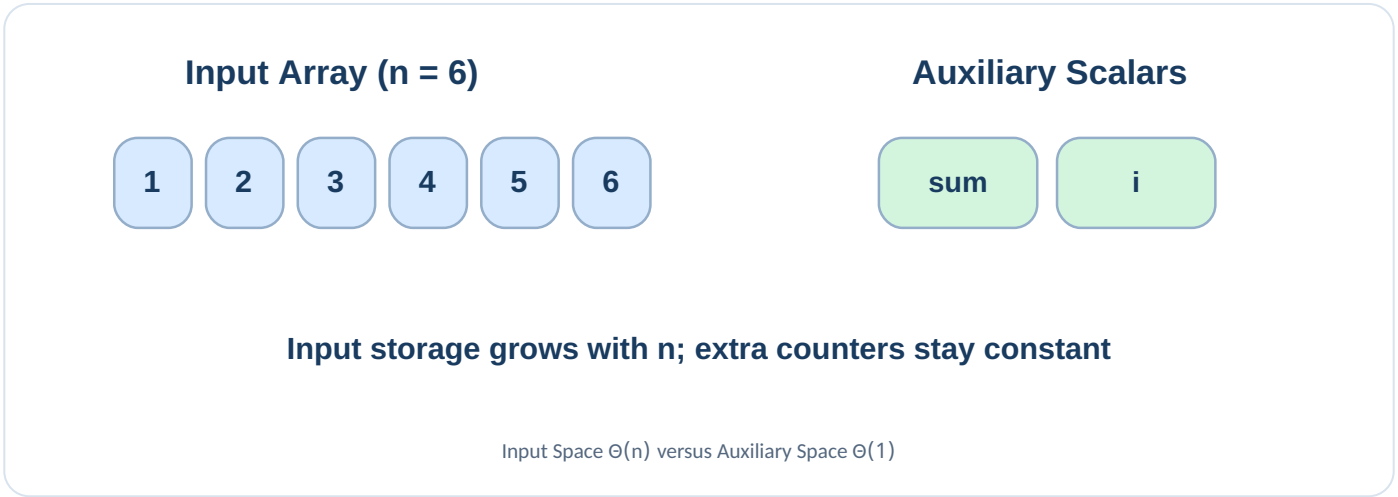
```
int sum_array(const int a[], int n) {
    int sum = 0;
    for (int i=0; i<n; ++i) sum += a[i];
    return sum;
}
```

- Input Array storage = `Θ(n)` (ধরা হচ্ছে integer-এর Size Fixed)।
- `sum`, `i`, `n`-এর মতো কয়েকটি Scalar = `Θ(1)` **Auxiliary Space**।
- Array-সহ **Total Space** = `Θ(n)`।

৫.২ Variable Memory: Copy Array

```
int *b = malloc(n * sizeof(int)); // ধারণাগত O(n) allocation
for (int i=0; i<n; ++i) b[i] = a[i];
free(b);
```

আরেকটি `n` Element-এর Array ⇒ `Θ(n)` Auxiliary Space; Copy Loop ⇒ `Θ(n)` Time। `malloc` ব্যর্থ হলে Check করতে হয়—উপরের Code শুধুমাত্র Complexity প্রদর্শনের জন্য।



Input Space Θ(n) versus Auxiliary Space Θ(1)

৫.৩ Arrays, Strings, Stack, Queue

Data / Operation	Typical Time	Extra Space / মন্তব্য
Fixed Array Index Access	`O(1)`	`O(1)`
Copy n-length Array	`O(n)`	`O(n)` for new copy
Mutable String Character Access	`O(1)` when random-access representation	`O(1)`
String Scan / Length Calculation	`O(n)` if scanning	`O(1)`
Stack Push/Pop	`O(1)` typical	stored stack items take `O(k)`
Queue Enqueue/Dequeue	`O(1)` typical	queue contents take `O(k)`
Hash Table Lookup	`O(1)` average, `O(n)` worst possible	Table may use `O(n)` storage

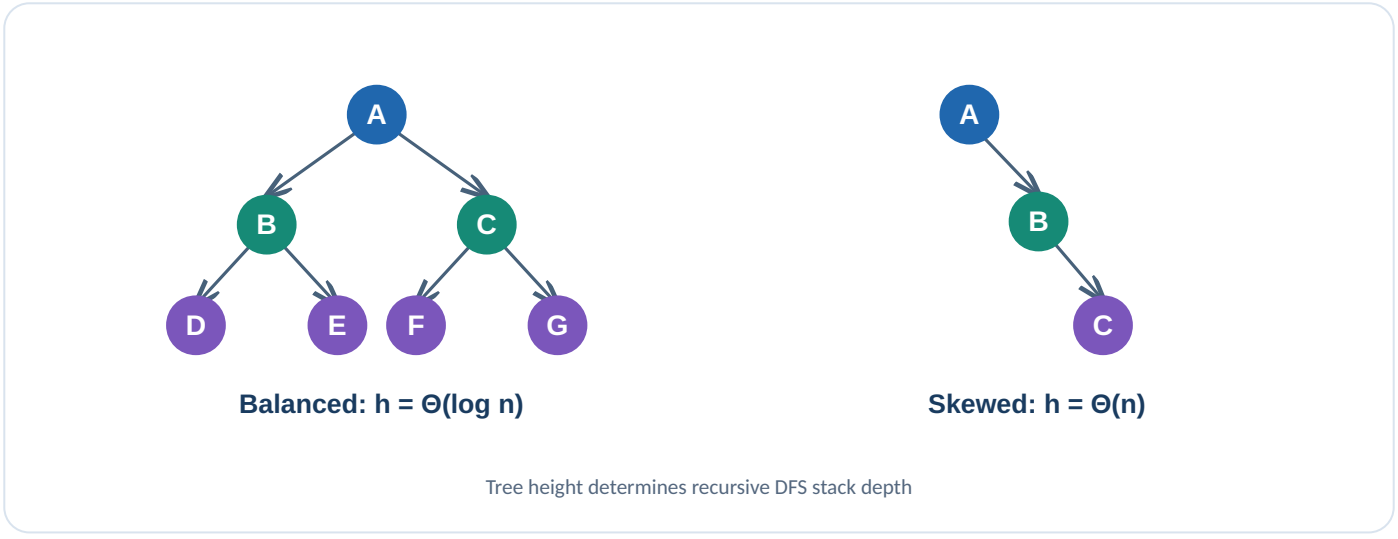
সতর্কতা: কোনো ভাষায় String Length cached থাকতে পারে (`O(1)`); অন্য Implementation-এ Scan করতে হতে পারে (`O(n)`)। Dynamic Array Append সাধারণত **Amortized O(1)**, একক Resize Operation `O(n)` হতে পারে।

৫.৪ Recursion Stack Visual

```
void f(int n) {
    if (n == 0) return;
    f(n - 1);
}
```

`f(4) → f(3) → f(2) → f(1) → f(0)` । Base Case-এ পৌঁছানোর মুহূর্তে সর্বাধিক ৫টি Stack Frame। n-এর সঙ্গে Frame Number লিনিয়ার ⇒ `O(n)` Stack Space। Tail-call Optimization **নিশ্চিত নয়**, তাই সাধারণ পরীক্ষার বিশ্লেষণে সেটি ধরে নেবেন না।

৫.৫ Balanced vs Skewed Tree, Height ও Width



Recursive DFS (Preorder/Inorder/Postorder): **Time `O(n)` , auxiliary stack `O(h)`** যেখানে `h` Tree Height (edge count হলে asymptotically frame count `h+1`)। Balanced Tree-তে `h=O(log n)` ; skewed Tree-তে `h=O(n)` ।

Level-order BFS: Queue-এর Space `O(w)`, যেখানে `w` Maximum Number of Nodes at One Level। একটি Balanced Complete Tree-এর শেষ Level বড় ⇒ `w=0(n)` হতে পারে। Skewed Tree-তে `w=1`, Queue Space `0(1)`। সর্বক্ষেত্রে Node Visit `0(n)` Time।

৫.৬ Iterative বনাম Recursive

Binary Search: Iterative Version: `0(log n)` Time, `0(1)` Extra Space। Recursive Version: `0(log n)` Time, `0(log n)` Stack Space (Tail-call Optimization না ধরে)। উভয় Algorithm-ই Sorted Array-তে সঠিক।

Tree DFS: Recursive Version Height অনুযায়ী Stack; Explicit Stack-ভিত্তিক Iterative DFS-এও Worst Case `O(h)` বা `O(n)` Storage পরিস্থিতি/Implementation অনুযায়ী হতে পারে।

৫.৭ In-place Algorithm

An in-place algorithm uses only a constant amount of auxiliary memory—এটি প্রচলিত ধারণা (কিছু Context-এ `O(log n)` Stack-সহ "in-place" বলা হয়; নির্দিষ্ট Definition স্পষ্ট রাখা ভালো)। Bubble Sort ও Selection Sort `0(1)` Auxiliary Space-এ Element Swap করতে পারে; Standard Merge Sort সাধারণত `0(n)` Extra Array ব্যবহার করে।

৫.৮ Time–Space Trade-off

- Hash Table-এ অতিরিক্ত Memory ব্যবহার করলে Search **Average** **`O(1)`**, কিন্তু Collision ও Resize-এর বিবেচনা আছে।
- Cached/Memoized Fibonacci অতিরিক্ত `O(n)` Memo Memory ব্যবহার করে `O(n)` Time অর্জন করতে পারে; Naive Recursive Version Exponential।
- Array Copy করলে Original Data অক্ষত থাকে, তবে `O(n)` অতিরিক্ত Storage লাগে।

Exam Tip: সবসময় দুটি আলাদা Line লিখুন: `Time = ...` এবং `Auxiliary Space = ...`।

৬. Important Algorithms: Search, Sort, Fibonacci, Array & String

> **Learning outcome:** Algorithm কীভাবে কাজ করে, Complexity কেন হয়, কোন Case ভিন্ন তা বিশ্লেষণ করতে পারবেন।

৬.১ Linear Search

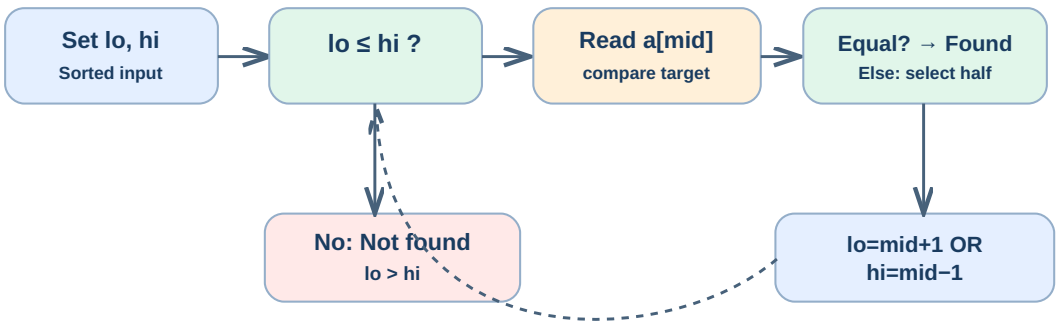
Idea: প্রথম Element থেকে একটির পর একটি Target Compare করা।

```
int linearSearch(const int a[], int n, int x) {
    for (int i=0; i<n; ++i)
        if (a[i] == x) return i;
    return -1;
}
```

- Best Time `O(1)` (প্রথম Element)।
- Average Time `O(n)` (সমান সম্ভাবনায় Target Position ধরলে)।
- Worst Time `O(n)` (শেষে/অনুপস্থিত)।
- Auxiliary Space `O(1)` ।

৬.২ Binary Search — Sorted Array আবশ্যক

Binary Search visual practice: Sorted Array [2,7,12,17,23,31,38,45,52], target 23 ⇒ first middle index=4, value=23 ⇒ found (best Θ(1)). Target 45 ⇒ 23 (move right) → 38 (move right) → 45 (found), worst Θ(log n).



Binary Search flowchart: compare middle, discard half, repeat until found / empty range

```
int binarySearch(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size()-1;
    while (lo <= hi) {
        int mid = lo + (hi-lo)/2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

প্রতি Step-এ অপ্রয়োজনীয় অর্ধেক বাদ। Best `0(1)`, Worst `0(log n)`, Iterative Auxiliary `0(1)`। Recursive Binary Search-এর Stack `0(log n)` Worst Case। `mid = lo+(hi-lo)/2` লেখা Overflow Risk কমায়।

৬.৩ Bubble Sort

Idea: পাশাপাশি দুটি Element Compare করে ভুল ক্রম হলে Swap করা; প্রতি Pass-এ বড় Element শেষের দিকে ওঠে। **Early-exit Flag** দিলে ইতিমধ্যে Sorted Input-এর Best Time `0(n)`।

```
void bubbleSort(int a[], int n) {
    for (int i=0; i<n-1; ++i) {
        int swapped = 0;
        for (int j=0; j<n-i-1; ++j) {
            if (a[j] > a[j+1]) {
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t;
                swapped=1;
            }
        }
        if (!swapped) break;
    }
}
```

Best `0(n)` (optimized), Average/Worst `0(n²)`; Auxiliary `0(1)`; **Stable**।

৬.৪ Selection Sort

Idea: বাকি Unsorted অংশ থেকে Minimum বের করে বর্তমান Position-এ Swap করা। প্রায় প্রতিটি Case-এ Comparisons = `n(n−1)/2`।

```
FOR i=0..n-2:
    minIndex = i
    FOR j=i+1..n-1:
        IF A[j] < A[minIndex]: minIndex = j
    SWAP A[i], A[minIndex]
```

Best/Average/Worst `0(n²)` Time, Auxiliary `0(1)`; Conventional Swapping Selection Sort সাধারণত **Unstable**।

৬.৫ Insertion Sort

Idea: প্রতিটি নতুন Element-কে ইতিমধ্যে Sorted Left Portion-এর সঠিক অবস্থানে Insert করা।

```
for (int i=1; i<a.length; i++) {
    int key = a[i];
    int j = i-1;
    while (j>=0 && a[j]>key) {
        a[j+1] = a[j];
        --j;
    }
    a[j+1] = key;
}
```

Sorted Input Best $O(n)$, Average/Worst $O(n^2)$; Auxiliary $O(1)$; **Stable**। Small/Nearly-sorted Input-এ কার্যকর।

> **Sorting demonstration:** Bubble Sort start [5,1,4,2,8] → swap(5,1) [1,5,4,2,8] → swap(5,4) [1,4,5,2,8] → swap(5,2) [1,4,2,5,8] → further passes [1,2,4,5,8]. Selection Sort minimum selection and Insertion Sort left sorted prefix are described in this chapter.

৬.৬ Merge Sort

```
MERGE_SORT(A)
    IF len(A) <= 1: RETURN A
    Split A into Left, Right
    RETURN MERGE(MERGE_SORT(Left), MERGE_SORT(Right))
```

$T(n)=2T(n/2)+\Theta(n)$ ⇒ Best/Average/Worst **$\Theta(n \log n)$**; Conventional Array Version Auxiliary **$\Theta(n)$**, Stack $O(\log n)$ (buffer space dominates); **Stable** with correct merge tie handling।

৬.৭ Quick Sort

Idea: Pivot দিয়ে Array-কে Partition; Left-এর ছোট Values এবং Right-এর বড় Values (scheme-dependent), দুই ভাগে Recursion।

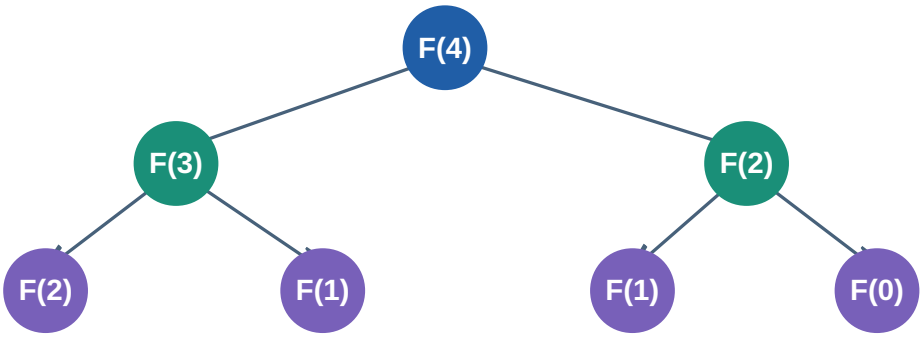
```
QUICKSORT(A, lo, hi)
    IF lo >= hi: RETURN
    p = PARTITION(A, lo, hi)
    QUICKSORT(A, lo, p-1)
    QUICKSORT(A, p+1, hi)
```

Balanced Partitions ⇒ $O(n \log n)$; Highly Unbalanced Partitions ⇒ $O(n^2)$ Worst। Typical randomized-pivot Quick Sort-এর **Expected** $O(n \log n)$ Time; In-place Partition সহ Recursive Stack **Expected** $O(\log n)$, Naive Worst $O(n)$। Conventional Quick Sort **Unstable**।

৬.৮ Fibonacci: Naive Recursive vs Iterative vs Memoization

```
int fibRecursive(int n) {
    if (n <= 1) return n;
    return fibRecursive(n-1) + fibRecursive(n-2);
}
```

Naive Recursive: Time $O(\varphi^n)$ (often upper-bounded by $O(2^n)$); Stack $O(n)$। Repeated Calls অনেক বেশি।



Repeated subproblems: F(2) and F(1) are recomputed

Partial recursive Fibonacci call tree (F(4)); subtrees omitted for brevity

```
long long fibIterative(int n) {
    if (n <= 1) return n;
    long long a=0, b=1;
    for (int i=2; i<=n; ++i) {
        long long next=a+b;
        a=b; b=next;
    }
    return b;
}
```

Iterative: Time `O(n)`, Extra Space `O(1)` (fixed-width integer ও arithmetic Cost O(1) ধরে; বড় n-এ Overflow হতে পারে)। Memoization: Time `O(n)`, Memo Storage `O(n)`, Recursive Stack `O(n)`।

factorial(4) stack: Call F(4) → F(3) → F(2) → F(1); return 1 → 2 → 6 → 24. Time Θ(n), recursive Stack Θ(n).

৬.৯ Array & String Operations

Operation	Typical Time	Auxiliary Space
Array Index Access	`O(1)`	`O(1)`
Array Traversal	`O(n)`	`O(1)`
Unsorted Array Linear Search	Best `O(1)`, Worst `O(n)`	`O(1)`
Insert at Front of Contiguous Array	`O(n)` shift	`O(1)` if capacity exists
Append to Dynamic Array	Amortized `O(1)`	Resizing can use `O(n)`
Reverse Mutable Array/String In-place	`O(n)`	`O(1)`
Copy String Length n	`O(n)`	`O(n)` for copy
Count Character Frequencies (fixed alphabet)	`O(n)`	`O(1)` for fixed alphabet
Build String by repeatedly copying immutable prefixes	`O(n²)` common naive pattern	Implementation-dependent

৬.১০ Algorithm Comparison Table (Most Tested)

Algorithm	Best Time	Average / Expected	Worst Time	Auxiliary Space	Stable?
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	N/A
Binary Search (iterative)	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$	N/A
Bubble Sort (optimized)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Yes
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Usually No
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Yes
Merge Sort (array)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Yes
Quick Sort (randomized pivot)	$\Theta(n \log n)$	Expected $\Theta(n \log n)$	$\Theta(n^2)$	Stack: expected $O(\log n)$, worst $O(n)$	Usually No
Fibonacci (naive recursion)	By n: exponential	By n: exponential	$\Theta(\varphi^n)$	$\Theta(n)$ stack	N/A
Fibonacci (iteration)	By n: $\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	N/A

Comparability Note: Fibonacci-এর Complexity-তে Input হলো `n` -এর Numeric Value; Bit-length of n বিবেচনা করলে Complexity Model বদলাবে। এখানে Conventional Introductory DSA Model ব্যবহার হয়েছে।

৭. Bangladesh Government IT Exam & Interview Preparation

> **Source disclosure:** নিচের প্রশ্নগুলো **এই গাইডের জন্য তৈরি অনুশীলনী**; যাচাইকৃত কোনো নির্দিষ্ট Bangladesh Government/Bank Past Paper হিসেবে দাবি করা হচ্ছে না।

৭.১ Written Exam-এর জন্য ছোট English Definitions

Term	Concise exam-ready definition
Algorithm	A finite sequence of precise steps for solving a problem.
Time complexity	Asymptotic growth of operations with input size.
Space complexity	Asymptotic growth of memory requirement with input size.
Auxiliary space	Extra working memory excluding the input storage.
Big O	An asymptotic upper bound.
Big Omega	An asymptotic lower bound.
Big Theta	An asymptotically tight bound.
Best case	Minimum cost among inputs of the same size.
Worst case	Maximum cost among inputs of the same size.
Recursion	A technique in which a function calls itself, subject to a base case.
In-place algorithm	An algorithm using constant additional storage under the strict definition.

৭.২ Short-answer Questions (Worked)

Q1. Why is Binary Search faster than Linear Search on large sorted arrays? Binary Search discards approximately half the remaining possibilities each step (`O(log n)` worst), whereas Linear Search may examine all n items (`O(n)` worst)। Sorted input এবং Direct Index Access প্রয়োজন।

Q2. What is the difference between `O(n)` and `Θ(n)` ? `O(n)` শুধু Upper Bound; `Θ(n)` একই সঙ্গে Upper ও Lower Bound। যেমন `1 = O(n)` কিন্তু `1 ≠ Θ(n)` ।

Q3. Why is Merge Sort `O(n log n)` ? প্রায় $\log_2 n$ স্তরের Divide Tree, প্রতিটি স্তরে মোট Θ(n) Merge Work; ফলে Θ(n log n) Time।

Q4. What is amortized complexity? Sequence of operations-এর মোট Cost ভাগ করে প্রতি Operation-এর গড় Guaranteed Bound; Dynamic Array Append-এর Amortized O(1), মাঝে মাঝে Resize O(n)। এটি Random Input-এর Average Case-এর সমার্থক নয়।

Q5. What is the difference between stack space and heap space? Recursion-related Active Function Frames সাধারণত Call Stack-এ থাকে; Dynamically allocated Arrays/Objects সাধারণত Heap/Managed Memory-তে থাকে। উভয়ই যথাযথ Space Analysis-এ বিবেচনা করতে হয়।

৭.৩ Code Output ও Calculation Practice

Problem 1

```
for (int i=0; i<4; i++) printf("%d", i);
```

Output: `0123` ; ৪টি Print। n কে Variable করলে Θ(n)।

Problem 2

```
int s=0;
for (int i=1; i<=4; i++) s+=i;
printf("%d",s);
```

Output: `10` ; n পর্যন্ত করলে Θ(n) Time, Θ(1) Auxiliary।

Problem 3

```
for (int i=1; i<=16; i*=2) printf("%d ",i);
```

Output: `1 2 4 8 16` ; Θ(log n) যদি Upper Limit n হয়।

Problem 4

```
int c=0;
for (int i=0; i<4; i++)
  for (int j=0; j<i; j++) c++;
printf("%d",c);
```

Output: `6` (0+1+2+3); General Time $\Theta(n^2)$।

Problem 5: `T(n)=2T(n/2)+n` — **Θ(n log n)**। প্রতিটি স্তরের কাজ n; স্তর log n।

Problem 6: `T(n)=T(n/2)+1` — **Θ(log n)**; Input প্রতি Call-এ অর্ধেক।

Problem 7: `for(i=0;i<n;i++) for(j=0;j<m;j++) work();` — **Θ(nm)**, দুটি independent Dimension।

Problem 8: `for(i=1;i*i<=n;i++) work();` — **Θ(√n)**, কারণ i চলে প্রায় √n পর্যন্ত।

Problem 9: DFS Traversal for n-node Tree — **Θ(n)** Time, **O(h)** Recursive Stack।

Problem 10: BFS Traversal for n-node Tree — **Θ(n)** Time, **O(w)** Queue।

৭.৪ MCQ Bank, Difficulty ও Interactive Practice Test

HTML Guide-এর **MCQ Practice** Tab-এ Beginner / Intermediate / Advanced নির্বাচন করে উত্তর দিন। সঙ্গে সঙ্গে সঠিক/ভুল, কারণ, Score এবং Reset পাবেন। PDF-এর শেষে সম্পূর্ণ Question Bank ও Answer Key সংযুক্ত আছে।

৭.৫ Interview-oriented Questions (Quick Responses)

1. **Why is O(n log n) often desirable for sorting?** Comparison-based sorting has an Ω(n log n) Worst-case Lower Bound in the comparison model; Merge Sort achieves Θ(n log n)।
2. **Can an O(n²) Algorithm outperform O(n log n)?** নির্দিষ্ট ছোট n এবং Constant Factor-এ পারে; Asymptotic Class একা Benchmark নয়।
3. **Why is Hash Lookup not guaranteed O(1)?** Worst-case Collision Pattern-এ many keys একই Bucket-এ পড়ে O(n) পর্যন্ত হতে পারে (Implementation-এর ওপর নির্ভর করে)।
4. **What is the memory cost of recursive Binary Search?** Θ(log n) Stack, unless TCO/Optimization assumption explicitly given।
5. **Does in-place always mean faster?** না; Memory কমতে পারে, কিন্তু Operation/Cache Behavior/Algorithm ভিন্ন।
6. **What if input is two dimensional?** n এবং m যথাযথভাবে ধরে `O(nm)` বা `O(V+E)`-এর মতো Bound লিখতে হবে।

৮. Final Revision Sheet — One-Page Exam Checklist

> **Before writing the answer:** Input Size চিহ্নিত করুন → Main Operation Count করুন → Worst/Average/Best Case আলাদা করুন → Dominant Term নিন → Auxiliary Space লিখুন।

৮.১ Formula Flashcards

Code Pattern / Idea	Time (Typical)	Memory Reminder
`a[k]`	`O(1)`	`O(1)` extra
`for i=0..n−1`	`O(n)`	loop scalar `O(1)`
`i *= 2`	`O(log n)`	`O(1)`
Two independent length-n loops	`O(n)`	`O(1)`
Fully nested n×n loops	`O(n²)`	`O(1)` if only scalars
Outer n, inner log n	`O(n log n)`	`O(1)` if only scalars
Triple nested n×n×n	`O(n³)`	`O(1)` if only scalars

Code Pattern / Idea	Time (Typical)	Memory Reminder
Array copy	`O(n)`	`O(n)` new array
Recursive linear chain	`O(n)`	`O(n)` stack
Balanced Tree DFS	`O(n)`	`O(log n)` stack
Skewed Tree DFS	`O(n)`	`O(n)` stack
Tree BFS	`O(n)`	`O(w)` queue
Binary Search iterative	`O(log n)` worst	`O(1)`
Merge Sort	`O(n log n)`	`O(n)`
Quick Sort	Expected `O(n log n)` randomized	Expected `O(log n)` stack

৮.২ Rapid Memory Hooks

- **Constant:** Input বড় হলেও Direct Access-এর ধাপ স্থির।
- **Linear:** প্রতিটি Element একবার।
- **Logarithmic:** প্রতিবার অর্ধেক / নির্দিষ্ট Ratio-তে কমে।
- **Quadratic:** প্রায় সব জোড়ার Comparison।
- **Recursive Space:** সর্বোচ্চ Simultaneously Active Call Count।
- **Time বনাম Space:** Operation Count আর Memory Count এক জিনিস নয়।
- **Big O বনাম Θ:** Upper Bound বনাম Tight Bound।
- **Exam Safeguard:** Worst/Expected/Amortized শব্দগুলো বাদ দেবেন না।

<details><summary>Suggested 30-minute Revision Plan</summary> ৫ মিনিট: Big O/Ω/Θ + Growth Ordering। ১০ মিনিট: ৮টি Loop/Recursion Pattern। ৫ মিনিট: ৭টি Sort/Search Algorithm Complexity Table। ১০ মিনিট: Advanced MCQ ও ভুলগুলো পুনরায় দেখুন। </details>

Appendix A — 46 Exam Practice MCQs (Original)

All questions are generated learning exercises, not independently verified past papers. Answers and explanations follow the questions.

Beginner: 15 Questions

1. **Algorithm বলতে কী বোঝায়?**
   (A) যেকোনো অসীম নির্দেশনা | (B) সসীম ও স্পষ্ট ধাপের সমস্যা সমাধান পদ্ধতি | (C) শুধুমাত্র Computer Program | (D) Computer-এর RAM
2. **Array-এর Element সংখ্যা 50 হলে n-এর মান কত?**
   (A) 5 | (B) 10 | (C) 50 | (D) 500
3. **Time Complexity মূলত কী বোঝায়?**
   (A) ফাইলের Size | (B) Input বাড়লে Operation-এর Growth | (C) Processor-এর দাম | (D) শুধু Second
4. **Space Complexity মূলত কী মাপে?**
   (A) Screen Brightness | (B) CPU Frequency | (C) Memory Requirement Growth | (D) Internet Speed
5. **Fixed Array Index Access-এর সাধারণ Complexity কী?**
   (A) $O(1)$ | (B) $O(n)$ | (C) $O(n^2)$ | (D) $O(2^n)$
6. **n বার চলা Single Loop-এর Time Complexity?**
   (A) $O(1)$ | (B) $O(n)$ | (C) $O(\log n)$ | (D) $O(n!)$
7. **দুটি পরপর n-Iteration Loop-এর Time Complexity?**
   (A) $O(n^2)$ | (B) $O(2n^2)$ | (C) $O(n)$ | (D) $O(\log n)$
8. **দুটি স্বাধীন Nested n-Loop-এর Complexity?**
   (A) $O(\log n)$ | (B) $O(n^2)$ | (C) $O(n)$ | (D) $O(1)$
9. **Binary Search চালানোর প্রধান Precondition কী?**
   (A) Unsorted Array | (B) Sorted Data | (C) Negative Numbers Only | (D) Duplicate Not Allowed
10. **Linear Search-এর Worst-case Time?**
    (A) $\Theta(\log n)$ | (B) $\Theta(1)$ | (C) $\Theta(n)$ | (D) $\Theta(n^2)$
11. **$O(\log n)$ কোন কাজের সঙ্গে সাধারণত মেলে?**
    (A) প্রতিবার Search Space অর্ধেক | (B) প্রতিটি Element দেখা | (C) সব Pair পরীক্ষা | (D) সব Permutation
12. **Auxiliary Space-এ কোনটি বাদ দেওয়া হয়?**
    (A) Loop Variable | (B) Temporary Buffer | (C) Input Storage | (D) Recursion Stack
13. **`$T(n)=7n+3$`-এর Tight Bound?**
    (A) $\Theta(n^2)$ | (B) $\Theta(n)$ | (C) $\Theta(1)$ | (D) $\Theta(\log n)$
14. **Stack Data Structure-এর মূল নীতি কী?**
    (A) FIFO | (B) LIFO | (C) Random Order | (D) Sorted Order
15. **Queue-এর নীতি কী?**
    (A) LIFO | (B) FIFO | (C) Binary Splitting | (D) Hashing

Intermediate: 15 Questions

1. **`` `for(i=1;i<n;i*=2)` ``-এর Time?**
   (A) $\Theta(n)$ | (B) $\Theta(n^2)$ | (C) $\Theta(\log n)$ | (D) $\Theta(1)$
2. **`$T(n)=3n^2+8n+1$`-এর Θ?**
   (A) $\Theta(n)$ | (B) $\Theta(n^2)$ | (C) $\Theta(n^3)$ | (D) $\Theta(\log n)$

3. `for(i=0;i<n;i++) for(j=0;j<m;j++)`-এর Time?
   (A) Θ(n+m) | (B) Θ(nm) | (C) Θ(n²) | (D) Θ(m²)
4. Triangular Loop-এর `0+1+...+(n−1)` কত?
   (A) n² | (B) n(n−1)/2 | (C) $\log_2 n$ | (D) n!
5. Balanced Binary Tree DFS Recursive Space?
   (A) O(1) | (B) O(n²) | (C) O(log n) | (D) O(2ⁿ)
6. Worst-case Skewed Tree DFS Stack?
   (A) O(1) | (B) O(log n) | (C) O(n) | (D) O(n²)
7. Tree Level-order Traversal-এর Data Structure?
   (A) Queue | (B) Stack Only | (C) Heap Only | (D) Hash Table Only
8. Optimized Bubble Sort-এর Best-case Time?
   (A) Θ(1) | (B) Θ(n) | (C) Θ(n log n) | (D) Θ(n²)
9. Selection Sort-এর Worst-case Time?
   (A) Θ(n) | (B) Θ(log n) | (C) Θ(n²) | (D) Θ(2ⁿ)
10. Merge Sort-এর Typical Auxiliary Space (Array)?
    (A) Θ(1) | (B) Θ(log n) | (C) Θ(n) | (D) Θ(n²)
11. Insertion Sort-এর Best Case?
    (A) Θ(n) | (B) Θ(n²) | (C) Θ(log n) | (D) Θ(1)
12. Recursive Binary Search-এর Stack Space (TCO না ধরে)?
    (A) Θ(1) | (B) Θ(log n) | (C) Θ(n) | (D) Θ(n²)
13. Two consecutive loops Θ(n) and Θ(n²) হলে মোট?
    (A) Θ(n³) | (B) Θ(n²) | (C) Θ(n) | (D) Θ(log n)
14. Big O কী প্রকাশ করে?
    (A) Tight Bound Only | (B) Lower Bound Only | (C) Asymptotic Upper Bound | (D) Exact Seconds
15. `n=16`; ধারাবাহিক অর্ধেক ১-এ পৌঁছাতে কত Halving?
    (A) 2 | (B) 4 | (C) 8 | (D) 16

Advanced: 16 Questions

1. `T(n)=2T(n/2)+n`-এর Tight Bound?
   (A) Θ(n) | (B) Θ(log n) | (C) Θ(n log n) | (D) Θ(n²)
2. `T(n)=T(n/2)+1`-এর Tight Bound?
   (A) Θ(log n) | (B) Θ(n) | (C) Θ(n log n) | (D) Θ(2ⁿ)
3. `f(n)=n` এবং `g(n)=n²`; কোনটি সত্য?
   (A) n = Θ(n²) | (B) n = o(n²) | (C) n = ω(n²) | (D) n² = O(n)
4. `f(n)=3n+5`-এর জন্য কোনটি সঠিক?
   (A) শুধু O(n) | (B) শুধু Ω(n) | (C) Θ(n) | (D) Θ(n²)
5. `for(i=1;i*i<=n;i++)`-এর Time?
   (A) Θ(log n) | (B) Θ(√n) | (C) Θ(n) | (D) Θ(n²)
6. `for(i=0;i<n;i++){while(j<n) j++;}` যেখানে j=0 outside — complexity?
   (A) Θ(n²) | (B) Θ(n log n) | (C) Θ(n) | (D) Θ(log n)
7. Naive Recursive Fibonacci-এর Time-এর Tight Order (fixed-cost arithmetic)?
   (A) Θ(n) | (B) Θ(n²) | (C) Θ(φⁿ) | (D) Θ(n!)
8. Naive Recursive Fibonacci-এর Stack Space?
   (A) Θ(1) | (B) Θ(log n) | (C) Θ(n) | (D) Θ(φⁿ)

9. **Randomized Quick Sort-এর Expected Time?**
   (A) $\Theta(n^2)$ | (B) $\Theta(n \log n)$ | (C) $\Theta(\log n)$ | (D) $\Theta(1)$
10. **Quick Sort-এর Worst-case Time?**
   (A) $\Theta(n \log n)$ | (B) $\Theta(n)$ | (C) $\Theta(n^2)$ | (D) $\Theta(2^n)$
11. **Amortized O(1) Append-এর অর্থ কী?**
   (A) প্রতিটি Append অবশ্যই O(1) | (B) Operation Sequence-এর Average Charge constant | (C) Average random input constant | (D) Worst সব Operation n^2
12. **Comparison-based sorting-এর Worst-case Lower Bound?**
   (A) $\Omega(n)$ | (B) $\Omega(\log n)$ | (C) $\Omega(n \log n)$ | (D) $\Omega(n^2)$
13. **একটি Sorted input-এর Binary Search best-case Cost?**
   (A) $\Theta(1)$ | (B) $\Theta(\log n)$ | (C) $\Theta(n)$ | (D) $\Theta(n \log n)$
14. **Hash Table Search-এর General Worst Case হতে পারে?**
   (A) $\Theta(1)$ সবসময় | (B) $O(\log \log n)$ সবসময় | (C) $O(n)$ | (D) $O(n!)$
15. **কোন বক্তব্যটি সঠিক?**
   (A) $O(n^2)$ algorithm কখনো $O(n \log n)$-এর চেয়ে দ্রুত না | (B) $\Theta(n)$ নিজে Time Unit | (C) Big O বাস্তব Runtime নিশ্চিত করে | (D) ছোট Input-এ Constant Factor গুরুত্বপূর্ণ
16. **Tree BFS Queue Space-এর প্রচলিত Bound?**
   (A) $O(h)$ | (B) $O(w)$ | (C) $O(\log n)$ সবসময় | (D) $O(1)$ সবসময়

Appendix B — Answer Key with Explanations

Beginner Answers

Q#	Answer	Explanation (বাংলায়)
1	B — সসীম ও স্পষ্ট ধাপের সমস্যা সমাধান পদ্ধতি	Algorithm হলো সমস্যা সমাধানের finite, unambiguous step sequence।
2	C — 50	এখানে Input Size হচ্ছে Array-এর দৈর্ঘ্য।
3	B — Input বাড়লে Operation-এর Growth	এটি Step/Operation Count-এর Growth Rate নির্দেশ করে।
4	C — Memory Requirement Growth	Memory-এর ব্যবহার Input Size-এর সঙ্গে কীভাবে বাড়ে তা মাপে।
5	A — O(1)	সরাসরি Index Access সাধারণত Constant Time।
6	B — O(n)	মোট n Iterations।
7	C — O(n)	$n+n=2n \Rightarrow \Theta(n)$।
8	B — O(n²)	$n \times n = n^2$ Iterations।
9	B — Sorted Data	Binary Search Ordered/Sorted structure-এর ওপর কাজ করে।
10	C — Θ(n)	সব Element পরীক্ষা করতে হতে পারে।
11	A — প্রতিবার Search Space অর্ধেক	প্রতিবার অর্ধেক করলে Steps logarithmically বাড়ে।
12	C — Input Storage	Auxiliary Space হলো Input বাদে অতিরিক্ত Working Memory।
13	B — Θ(n)	Leading Term হলো 7n।
14	B — LIFO	Stack Last-In-First-Out নিয়মে কাজ করে।
15	B — FIFO	Queue First-In-First-Out নিয়মে কাজ করে।

Intermediate Answers

Q#	Answer	Explanation (বাংলায়)
1	C — Θ(log n)	1,2,4,8,... এভাবে i দ্বিগুণ হয়।
2	B — Θ(n²)	Dominant Term n^2।
3	B — Θ(nm)	Dimension দুটি স্বাধীন হওয়ায় n×m।
4	B — n(n−1)/2	Arithmetic Series Formula।
5	C — O(log n)	Balanced Tree Height Θ(log n), Recursive Stack O(h)।
6	C — O(n)	Height n−1 পর্যন্ত হতে পারে।
7	A — Queue	BFS-এর জন্য Queue ব্যবহৃত হয়।
8	B — Θ(n)	Sorted Input-এ একটি pass শেষে swap না হলে থামে।
9	C — Θ(n²)	সব Case-এ প্রায় n(n−1)/2 Comparisons।
10	C — Θ(n)	Standard Merge Buffer-এ n Element Storage লাগে।
11	A — Θ(n)	Already-sorted Array-তে একটি Linear Pass লাগে।
12	B — Θ(log n)	Recursive Depth log n।
13	B — Θ(n²)	$n+n^2$-এর Leading Term n^2।

Q#	Answer	Explanation (বাংলায়)
14	C — Asymptotic Upper Bound	Big O Upper Bound; Case (best/worst) আলাদা বিষয়।
15	B — 4	16→8→4→2→1; মোট 4টি Halving।

Advanced Answers

Q#	Answer	Explanation (বাংলায়)
1	C — $\Theta(n \log n)$	প্রতি Level-এর কাজ n, Levels log n।
2	A — $\Theta(\log n)$	একটি Half Problem-এ Recursion হয়।
3	B — $n = o(n^2)$	$n/n^2 = 1/n \to 0$।
4	C — $\Theta(n)$	Upper ও Lower দুই Bound-ই linear।
5	B — $\Theta(\sqrt{n})$	i সর্বোচ্চ প্রায় √n পর্যন্ত যায়।
6	C — $\Theta(n)$	j সব Outer Iteration মিলিয়ে n বার বাড়ে, Reset হয় না।
7	C — $\Theta(\varphi^n)$	Call Tree Fibonacci Growth; $\varphi \approx 1.618$।
8	C — $\Theta(n)$	সবচেয়ে গভীর Call Chain n দৈর্ঘ্যের।
9	B — $\Theta(n \log n)$	Random Pivot Balanced Partition-এর প্রত্যাশিত Cost n log n।
10	C — $\Theta(n^2)$	প্রতিবার n−1 ও 0 ভাগে Partition হলে quadratic।
11	B — Operation Sequence-এর Average Charge constant	Resize O(n) হলেও অনেক Append মিলিয়ে প্রতি Append-এর amortized Cost O(1)।
12	C — $\Omega(n \log n)$	Comparison Decision Tree-তে অন্তত $\log_2(n!) = \Omega(n \log n)$ Comparisons দরকার।
13	A — $\Theta(1)$	প্রথম Middle Element-এই পাওয়া গেলে একটি comparison।
14	C — O(n)	Collision/Implementation-এর কারণে বহু Item Scan হতে পারে।
15	D — ছোট Input-এ Constant Factor গুরুত্বপূর্ণ	Asymptotic Bound বাস্তব Hardware/Constants প্রকাশ করে না।
16	B — O(w)	সর্বাধিক এক Level Width-এর সঙ্গে Queue-এর Size বৃদ্ধি পায়।